

Analysis of Road Representations in Search-Based Testing of Autonomous Driving Systems

Ezequiel Castellano
National Institute of Informatics
Tokyo, Japan
ecastellano@nii.ac.jp

Ahmet Cetinkaya
National Institute of Informatics
Tokyo, Japan
cetinkaya@nii.ac.jp

Paolo Arcaini
National Institute of Informatics
Tokyo, Japan
araini@nii.ac.jp

Abstract—Validating Autonomous Driving Systems (ADSs) is essential to ensure that the ADS meets the necessary requirements to be widely accepted. Simulation-based testing is one of the main validation approaches, in which the ADS is run in a simulated environment over different scenarios. In this context, search-based testing (SBT) is used to generate scenarios that possibly expose particular failures of the ADS under test. Most SBT approaches search for behaviors of other traffic participants, but usually fix the road map of the scenario in advance. Recently, the SBT community started investigating the search for road structures, which is particularly useful when testing specific components of the ADS, such as the lane-keeping component. However, roads can be represented in multiple ways and the impact of using a particular representation on the effectiveness of SBT is unclear. To fill this gap, this paper investigates the usage of six road representations for SBT of ADSs. As a representative SBT approach, we test the lane-keeping component of an ADS in the BeamNG.tech simulator, aiming to generate roads in which the autonomous vehicle drives off the lane. We study the effectiveness of each road representation in terms of triggered failures and also diversity of the generated roads.

Index Terms—Autonomous Driving Systems, Search-Based Testing, Simulation-Based Testing, Road Representation

I. INTRODUCTION

Automated and Autonomous Driving Systems (ADSs) have the potential of drastically changing the transportation sector, promising several benefits such as reduction in the number of accidents, lower gas emissions, less congestion, etc. [39] The ADS industry is growing rapidly and working towards the deployment of this technology. However, reports of collisions of self-driving cars [27], [20] show that there is still need for more extensive validation of the technology, before it is widely accepted and massive deployment is possible.

Regarding ADS validation, it has been shown that testing the ADS by running an autonomous car in the real world is not only dangerous and expensive, but also not very efficient, as one would need to drive billions of miles to have some confidence on the reliability of the ADS; namely, 11 billion miles should be driven to have 95% confidence that the autonomous vehicle has a better failure rate than a human driver [23]. Indeed, most on-road drives do not include situations which are particularly interesting from a testing perspective. To overcome this limitation, *simulation-based testing* [7] is a popular approach that leverages the

variety of scenarios in which an ADS can be tested, avoids the risks of having real-world accidents, and reduces costs.

In simulation-based testing, test designers can specify *test goals* (also called *test requirements*), and then try to generate test scenarios that *cover* them. Some test goals may simply require to observe a particular traffic and environmental condition in the scenario (e.g., the car must cross an intersection), and so they are relatively easy to cover. Other scenarios, instead, may require to observe particular driving behaviors (e.g., the car must turn right abruptly with high lateral acceleration, possibly to avoid a collision), which are more difficult to cover. Indeed, since the search space of scenarios is almost infinite, finding a scenario that triggers the ADS behavior required by the test goal is nontrivial. Search-based testing (SBT) [30] is a powerful technique, based on meta-heuristic search, that is particularly effective in ADS testing [12], [10], [35], [13], [28], [26]. This technique uses an objective function to guide the search towards scenarios that cover specific test goals.

The larger the search space is, the harder the search becomes. Therefore, since searching for complete scenarios is not feasible, most SBT approaches for testing ADS only search for specific characteristics of a scenario [8], [13], [28], such as the initial position, velocity, and acceleration of some traffic participants (e.g., vehicles or pedestrians), but they typically fix the road map of the scenario in advance.

However, in order to cover particular test goals or to test particular ADS components (e.g., the lane-keeping component), the road structure is the most important feature of the scenario and thus should be the actual target of the search. Recently, some works started investigating the search for roads [34], [22], [16], [24] in SBT of ADSs. Roads can be represented in several ways and different works use different representations. For example, Riccio and Tonella [34] adopt Catmull-Rom spline representation, Gambi et al. [22] use parametric straight and curved road segments to construct road networks, Castellano et al. [16] represent roads using curvature values as in Frenet frames, while Klück et al. [24] use Bézier curves. The previously mentioned works adopt a specific representation without any particular justification; as a matter of fact, in the literature there is not much knowledge about the effect that each representation may have on the effectiveness of SBT. For example, one road representation could allow to obtain sharper curvatures than another one, and

this could affect the effectiveness of the search algorithm in optimizing the objective functions.

In this paper, we aim to fill this gap by studying a set of representative road representations, that identify different ways to define a road and that we believe are worthy to be investigated in the context of SBT of ADSs. In particular, we investigate how road representations affect the effectiveness of SBT when testing a lane-keeping component of an ADS, which heavily relies on the shape of the road. The goal is to find roads in which the vehicle can not keep the lane. The search is conducted by *RoadSearch*, a general search framework with standard mutation and crossover operators based on *Frenetic* [16], a search approach that combines exploration and exploitation to find a diverse test suite; we adopt this framework, as it has been shown to be very effective in testing the lane-keeping component [33]. The framework is instantiated with six different road representations, namely 1) *Cartesian_{Cat}*: Catmull-Rom splines with Cartesian control nodes, 2) *Cartesian_{Bez}*: Bézier curves with Cartesian control nodes, 3) *Kappa*: curvature-based representation, 4) *Kappa_{Step}*: curvature-based representation with varying segment lengths, 5) *Theta*: orientation-based representation, 6) *Theta_{Step}*: orientation-based representation with varying segment lengths. As for the simulation engine, we use *BeamNG.tech* [6], which is a research oriented simulator with a very accurate driving physics engine that has been used for ADS testing in other works [22], [34], [33].

We perform a set of experiments with the aim of discovering which road representation is more *effective* in SBT for ADS, using the testing of the lane-keeping component as representative case study. Specifically, we assess *which road representation allows to find more failures* (RQ1.a), and *which one produces the most diverse roads* (RQ1.b). Moreover, we assess the effect of mutations and crossover on different representations. We study, among different representations, *which is the impact of mutations, regardless of the position where the mutation is done* (RQ2.a) and also, within a given representation, *which is the impact of mutations when applied in different positions of the individual* (RQ2.b). As for crossover, we evaluate *how different are the children produced by crossover from their parents in each representation* (RQ3).

Paper structure. Sect. II introduces the details on the different representations of roads considered in the study. Sect. III presents the search algorithm and its instantiation with the road representations. Sect. IV introduces the research questions to be addressed and our experimental design. Sect. V presents the results of the experiments. Sect. VI discusses some threats that may affect the validity of the study and how we tried to mitigate them. Sect. VII surveys related work. Sect. VIII presents the main conclusions and outlines future work.

II. CONSIDERED ROAD REPRESENTATIONS

In this section, we provide mathematical preliminaries on the characterization of different road representations that we consider in our study. Specifically, we explain planar curves

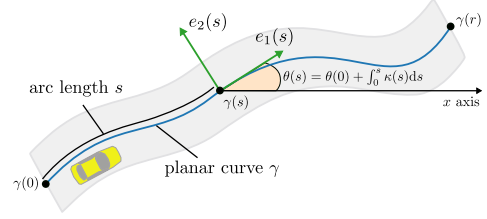


Fig. 1: Planar curve γ , Frenet frame $(e_1(s), e_2(s))$, and orientation θ as the integral of curvature κ

and how they are used for representing roads in Sect. II-A. We then look at distance functions that allow us to quantify the similarity/difference between different roads in Sect. II-B.

A. Road representations based on planar curves

We start with a short overview of planar curves, which we use as a common basis of our different road representations. In our characterization of curvature and orientation of curves, we rely on Frenet frames [25], which are used in trajectory planning for self-driving systems (see, e.g., [36], [40]).

Let $\gamma: [0, r] \rightarrow \mathbb{R}^2$ be a smooth planar curve of length $r > 0$, parameterized by arc length. Now, let $e_1(s) \in \mathbb{R}^2$ denote the unit tangential vector along the curve at arc length $s \in [0, r]$, and moreover, let $e_2(s)$ be the unit normal vector obtained by 90-degree rotation of $e_1(s)$ in counterclockwise direction. The pair $(e_1(s), e_2(s))$ is called the *Frenet frame* [25] of γ at s . Fig. 1 shows an example planar curve γ together with the associated Frenet frame.

The *curvature* $\kappa: [0, r] \rightarrow \mathbb{R}$ is identified as the unique function that satisfies $de_1(s)/ds = \kappa(s)e_2(s)$. Furthermore, *orientation* $\theta: [0, r] \rightarrow \mathbb{R}$ of the curve is given by

$$\theta(s) = \theta(0) + \int_0^s \kappa(s) ds, \quad (1)$$

where $\theta(0)$ is the initial orientation of the curve. It follows from [25, Theorem 1.2.2], that κ completely determines γ up to translation and rotation. In particular, $\gamma(s)$ satisfies

$$\gamma(s) = \gamma(0) + \int_0^s \begin{bmatrix} \cos \theta(s) \\ \sin \theta(s) \end{bmatrix} ds, \quad (2)$$

where $\gamma(0) \in \mathbb{R}^2$ is the starting position of the curve.

Curves associated with roads allow a variety of *parametric* and *nonparametric* representations. In parametric representations, coordinates of the points on a curve can be written as functions of a single parameter and different points on the curve correspond to different values of this parameter. In the following, we introduce the six parametric/nonparametric representations that are considered in this study: 1) Catmull-Rom splines with Cartesian control nodes, 2) Bézier curves with Cartesian control nodes, 3) curvature-based representation, 4) curvature-based representation with varying segment lengths, 5) orientation-based representation, and 6) orientation-based representation with varying segment lengths.

We consider Catmull-Rom splines as a representative of parametric curve generation approaches that are based on

interpolation of a number of control nodes. We also adopt a similar parametric approach that is based on Bézier curves. On the other hand, curvature-based and orientation-based representations are nonparametric representations. Curvature and orientation profiles provide information about the turns and angular variations in the curves.

While the particular structures of the considered representations are different, they all allow us to obtain a sequence of points $\gamma_0, \gamma_1, \dots, \gamma_{N-1} \in \mathbb{R}^2$ that characterize positions on a planar curve. For each representation, the obtained points $\gamma_0, \gamma_1, \dots, \gamma_{N-1} \in \mathbb{R}^2$ are interpolated and then passed to the BeamNG.tech driving simulator [6] as the Cartesian coordinates of road points.

1) *Catmull-Rom Splines with Cartesian control nodes* (Cartesian_{Cat}): Our first representation is based on Catmull-Rom splines [17]. In its construction, we consider a sequence of control nodes $P_0, P_1, \dots, P_n \in \mathbb{R}^2$. The initial node is fixed at $P_0 = (0, 0)$ and the other nodes are given as

$$P_i = P_{i-1} + \begin{bmatrix} \Delta_{x_{i-1}} \\ \Delta_{y_{i-1}} \end{bmatrix},$$

where the pairs $(\Delta_{x_0}, \Delta_{y_0}), (\Delta_{x_1}, \Delta_{y_1}), \dots, (\Delta_{x_{n-1}}, \Delta_{y_{n-1}})$ represent the increments in Cartesian coordinates. Given these pairs, Catmull-Rom interpolation generates a curve that passes through $P_0, P_1, \dots, P_n$. Then, with N samples obtained from this curve, we get $\gamma_0, \gamma_1, \dots, \gamma_{N-1}$.

2) *Bézier curves with Cartesian control nodes* (Cartesian_{Bez}): The representation is also a sequence of control nodes as in Cartesian_{Cat} , but uses Bézier interpolation to produce smoother curves. A Bézier curve always starts at P_0 and finishes at P_n , but, opposite to Catmull-Rom splines, rarely passes through the intermediate control nodes $P_1, \dots, P_{n-1}$.

3) *Curvature-based representation (Kappa)*: Our next road representation is based on curvatures. In particular, we consider N curvature values $\kappa_0, \kappa_1, \dots, \kappa_{N-1}$ and obtain $\gamma_0, \gamma_1, \dots, \gamma_{N-1}$ by (1) and (2) through integration with trapezoidal rule.

Suppose the curvature values $\kappa_0, \kappa_1, \dots, \kappa_{N-1}$ correspond to arc lengths $s_0 = 0, s_1, \dots, s_{N-1}$. We set $\theta_0 = \pi/2$ and obtain the orientation values $\theta_1, \dots, \theta_{N-1}$ by

$$\theta_{i+1} = \theta_i + \frac{\kappa_{i+1} + \kappa_i}{2} \Delta_{s_i}, \quad (3)$$

where $\Delta_{s_i} = s_{i+1} - s_i$ for $i \in 0, 1, \dots, N-2$. Next, after setting $\gamma_0 = (0, 0)$, we compute $\gamma_1, \dots, \gamma_{N-1}$ by

$$\gamma_{i+1} = \gamma_i + \frac{1}{2} \left(\begin{bmatrix} \cos \theta_{i+1} \\ \sin \theta_{i+1} \end{bmatrix} + \begin{bmatrix} \cos \theta_i \\ \sin \theta_i \end{bmatrix} \right) \Delta_{s_i}. \quad (4)$$

In this construction, the curve γ is composed of line segments. We consider a fixed segment length $\tilde{s} \in (0, \infty)$ and set $\Delta_{s_i} = \tilde{s}$ for every $i \in 0, \dots, N-2$. If the total curve length r is given, we have $\tilde{s} = r/(N-1)$. Fig. 2 shows θ_i and γ_i for a given sequence of curvature values $\kappa_0, \kappa_1, \dots, \kappa_{N-1}$.

The next representation considers varying segment lengths.

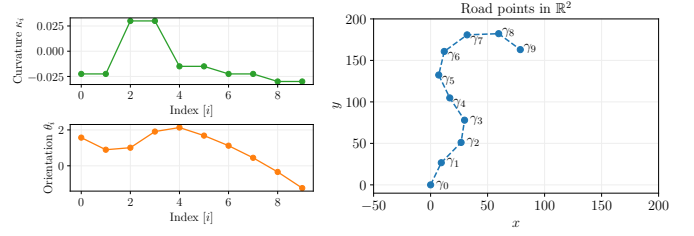


Fig. 2: Road orientations θ_i and positions γ_i for given curvature values κ_i

4) *Curvature-based representation with varying segment lengths (Kappa_{Step})*: We extend the previous curvature-based road representation by considering nonuniform segmentation of the curve. In particular, given the curvature values $\kappa_0, \kappa_1, \dots, \kappa_{N-1}$ and the segment lengths $\Delta_{s_0}, \Delta_{s_1}, \dots, \Delta_{s_{N-2}}$, we set $\gamma_0 = (0, 0)$ and obtain $\gamma_1, \dots, \gamma_{N-1}$ based on (3) and (4).

5) *Orientation-based representation (Theta)*: Curvature-based representations that we discussed previously consider orientation values as an intermediate representation for computing the positions on a curve. Another approach is to represent the roads in a more direct manner by considering orientation increments. In particular, given increments $\Delta_{\theta_0}, \Delta_{\theta_1}, \dots, \Delta_{\theta_{N-2}}$, we set $\theta_0 = \pi/2$ and compute $\theta_{i+1} = \theta_i + \Delta_{\theta_i}$ for $i \in 0, \dots, N-2$. With this computation, we obtain $\theta_0, \theta_1, \dots, \theta_{N-1}$. Then, after setting $\gamma_0 = (0, 0)$, we obtain $\gamma_1, \dots, \gamma_{N-1}$ by using (4) with a fixed segment length ($\Delta_{s_i} = \tilde{s}$). While the segment length is used in calculation of γ_i , this representation omits the segment length in orientation calculation by replacing the integration step (3) with simple increments. This change in the representation affects the results that we obtain.

6) *Orientation-based representation with varying segment lengths (Theta_{Step})*: Orientation-based road representation explained above can be extended by considering nonuniform segmentation of the curve. The computation of θ_i remains the same. However, instead of fixing a uniform segment length, we consider different segment lengths $\Delta_{s_0}, \Delta_{s_1}, \dots, \Delta_{s_{N-2}}$. Again we set $\gamma_0 = (0, 0)$ and obtain $\gamma_1, \dots, \gamma_{N-1}$ by using (4) with these segment lengths.

B. Distance between curves that represent roads

In our work, we are interested in assessing which types of roads are generated with SBT using the different road representations. In particular, we are interested in assessing the *diversity* of the generated roads, as this is an indication of the exploration of the search space. Therefore, we need a way to compare two roads.

Distance measures are useful for comparing roads. Specifically, we can quantify the similarity/difference between two roads by computing the “distance” between the curves associated with those roads. The distance measures that we consider in this work are particularly suited to our setting.

In our problem setting and for the particular driving agent that we consider, neither the location nor the orientation of a

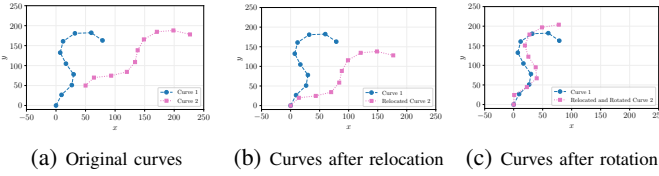


Fig. 3: Relocation and rotation steps to “align” curves before computing their discrete Fréchet distance

road in the map affects the driving behavior. On the other hand, the curvature profile of a road is important and the performance of the driving agent depends on how the road curves along its length. Based on this observation, we compare roads by using *discrete Fréchet distance* [32], after some processing steps based on *Procrustes analysis* [18], which we explain below.

1) *Discrete Fréchet distance*: One of the most common approaches for quantifying the similarity/difference between two curves is to use the Fréchet distance [5]. Given two curves $\gamma: [0, r] \rightarrow \mathbb{R}^2$ and $\psi: [0, q] \rightarrow \mathbb{R}^2$, Fréchet distance $d_F(\gamma, \psi)$ is defined by

$$d_F(\gamma, \psi) = \inf_{\alpha \in \mathcal{F}_r, \beta \in \mathcal{F}_q} \max_{t \in [0, 1]} \|\gamma(\alpha(t)), \psi(\beta(t))\|,$$

where $\mathcal{F}_s$ (for a given $s > 0$) denotes the set of continuous and non-decreasing functions from $[0, 1]$ to $[0, s]$. When the curves γ and ψ are composed of line segments between a finite number of points $\gamma_0, \gamma_1, \dots, \gamma_{N-1} \in \mathbb{R}^2$ and $\psi_0, \psi_1, \dots, \psi_{M-1} \in \mathbb{R}^2$, the Fréchet distance between them can be approximated closely by the so-called *discrete Fréchet distance* $d_{DF}(\gamma, \psi)$, which allows efficient computation (see [32], [1]).

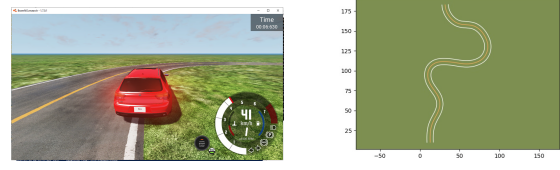
2) *Relocation and Procrustes rotation steps*: In this work, we use the discrete Fréchet distance after certain processing steps. In particular, given two curves that represent roads, we “align” the curves through relocation and rotation.

First, we relocate one of the curves in the map so that the initial points of both curves overlap (see Fig. 3b). As the initial points represent the locations from which the car starts moving, this relocation step allows us to take account of the driving behavior when assessing the difference of two roads. The second step is the rotational Procrustes analysis [18], that allows us to superimpose the curves with respect to certain optimality criteria (see Fig. 3c). Specifically, given the points $\gamma_0, \gamma_1, \dots, \gamma_{N-1} \in \mathbb{R}^2$ and $\phi_0, \phi_1, \dots, \phi_{M-1} \in \mathbb{R}^2$ representing two curves γ and ϕ that start from the same point ($\gamma_0 = \phi_0$), we rotate ϕ along the initial point ϕ_0 to obtain a rotated curve ψ so that $\sum_{i=0}^{\min\{N, M\}-1} \|\gamma_i - \psi_i\|^2$ is minimized. The optimal rotation angle ϑ achieving this task is given by

$$\vartheta = \arctan \left(\frac{\sum_{i=0}^{\min\{N, M\}-1} (\phi_{i,1}\gamma_{i,2} - \phi_{i,2}\gamma_{i,1})^2}{\sum_{i=0}^{\min\{N, M\}-1} (\phi_{i,1}\gamma_{i,1} + \phi_{i,2}\gamma_{i,2})^2} \right).$$

We note that relocation and rotation steps do not change the curvature of the roads, thus the driving behaviors obtained for these roads remain the same.

After the rotational Procrustes analysis, the difference between the aligned curves (representing two roads) is quantified



(a) BeamNG.tech simulator (b) One-way road (200x200 map)

Fig. 4: ADS simulator

by computing the discrete Fréchet distance. Note that since discrete Fréchet distance is sensitive to road lengths, we normalize it by dividing it by the length of the longest road: we call it *normalized discrete Procrustes-Fréchet* d_{NDPF} .

III. PROPOSED APPROACH

In this section, we propose a framework to assess the influence of road representation in search-based testing (SBT) of autonomous driving systems (ADSs). In Sect. III-A, we present an ADS whose testing requires the generation of different types of road. In Sect. III-B, we show how we encode the different road representations introduced in Sect. II-A as individuals of a search algorithm. We present in Sect. III-C a general search framework, for testing the ADS under study, that can be instantiated with the different road representations.

A. ADS under test

To assess how the road representation affects SBT for ADS, we need to select an ADS component whose behavior is highly influenced by the road structure. To this aim, we select, as target system to be tested, the lane-keeping component of an autonomous vehicle of the BeamNG.tech simulator [6] shown in Fig. 4a. Given a map size of S meters and a time budget T_b , the goal is to generate one-way roadways (as the one shown in Fig. 4b) that fit in a map of size $S \times S$ and make the lane-keeping component fail. The goal of the car is to drive from the beginning to the end of the road. A *failure* is defined as making the car be out of bounds, with a predefined criterion about the percentage of the car that needs to be out of the road to be considered a failure. Given that there are no other traffic participants involved, failures can only be triggered by the road shape. In our experiments, we use tolerance 0.95, which means that almost the entire car has to be out of the lane to be considered a failure. Besides, the car model has a restriction of 70 km/h on the maximum speed to avoid triggering failures due to unrealistic speeds. The values of these two parameters are also used in [33] to make the search problem difficult. The size of the map is defined as 200m x 200m to obtain interesting but short segments, which is usually of interest when producing a test suite.

A *test* is defined as a list of points $(x_0, y_0), \dots, (x_{N-1}, y_{N-1}) \in (S \times S)^N$ that define the center line of the road, as shown in Fig. 1. These points are then interpolated in order to produce a smoother road. A test is said to be *valid* if the interpolated road is within the map boundaries, is not self-intersecting, and the curves are not too sharp. A valid test

is then passed on to the simulator, which returns an outcome (failure or pass). In addition, the simulator provides metrics about the drive, such as speed, acceleration, and location of the autonomous vehicle over time. In particular, we are interested in the out of bounds distance (*oob distance*), which measures the shortest distance between the center of mass of the car and either of the lines defining the boundaries of the driving lane. The driving lane has a width of 4 meters. Thus, an *oob distance* of 2.0 indicates that the car is in the center of the lane, a value of 0.0 indicates that the center of mass is on either of the lines that denote the driving lane, and a negative value indicates that the center of mass of the car has crossed a line delimiting the lane.

B. Road representations

Search-based approaches represent the *individuals* (i.e., possible solutions of the search problem) using particular *encodings*, as sequences of *search variables* (also called *genes*). The most straightforward way to represent roads would be by encoding the absolute coordinates of the points in the plane. However, using absolute position values would not be an effective representation for operations such as crossover, which are used in search algorithms. For instance, the swapping performed in a chromosome crossover could lead to the creation of two consecutive points of the representation very far from each other, so not representing a valid road. Thus, all the approaches considered here adopt a *relative encoding*, i.e., each element of the representation is defined relatively to the previous element. Cartesian_{Bez} and Cartesian_{Cat} are represented as a list of tuples (Δ_x, Δ_y) , which are the relative position w.r.t. to the previous point. Theta is represented as a list of angular differences in radians w.r.t. to the previous angles that are then interpolated assuming a uniform segment length. Kappa is represented as a list of curvature values and also assumes a uniform segment length. In addition, we evaluate Kappa and Theta with variable segment length, i.e., the segment length is part of the gene encoding. To summarize, the representations of an individual are as follows:

- Cartesian_{Bez}: $[(\Delta_{x_0}, \Delta_{y_0}), \dots, (\Delta_{x_n}, \Delta_{y_n})]$
- Cartesian_{Cat}: $[(\Delta_{x_0}, \Delta_{y_0}), \dots, (\Delta_{x_n}, \Delta_{y_n})]$
- Kappa: $[\kappa_0, \dots, \kappa_n]$
- Kappa_{Step}: $[(\kappa_0, \Delta_{s_0}), \dots, (\kappa_{n-1}, \Delta_{s_{n-1}}), (\kappa_n, -)]$
- Theta: $[\Delta_{\theta_0}, \dots, \Delta_{\theta_n}]$
- Theta_{Step}: $[(\Delta_{\theta_0}, \Delta_{s_0}), \dots, (\Delta_{\theta_n}, \Delta_{s_n})]$

Each representation is then transformed into absolute Cartesian points to be evaluated in the simulator. The details of these transformations have been explained in Sect. II.

1) *Generating a random test*: A search-based approach usually needs to randomly generate individuals at the beginning of the search. In our context, to randomly generate individuals, each representation follows a sequential approach starting from the first gene, with certain restrictions on the values that it can generate for the next gene.

For Cartesian_{Cat} and Cartesian_{Bez}, a gene $(\Delta_{x_i}, \Delta_{y_i})$ is generated based on the location of the previous point (x_{i-1}, y_{i-1}) , a fixed segment length and an angle α within

a range defined by the max angle β parameter (i.e., $-\beta \leq \alpha \leq \beta$). Additionally, these configurations have a parameter to define the number of points that are used in the interpolation.

For Kappa, the value of a gene κ_i is obtained from a range defined by a global bound and a relative bound. The global bound (gb^-, gb^+) defines the minimum and the maximum value that κ_i can take, while the relative bound rb defines the difference between κ_i and its previous value κ_{i-1} (i.e., $\max(gb^-, \kappa_{i-1} - rb) \leq \kappa_i \leq \min(\kappa_{i-1} + rb, gb^+)$).

Kappa_{Step} generates κ values in the same way Kappa does. As for the segment length, it selects a value from a range defined by the *mean segment length* sl and the *segment variance* sv (i.e., $\max(0.5, sl - sv) \leq \Delta_s \leq sl + sv$), where sl and sv are hyper-parameters of the approach. Note that the segment values are one less than the kappa values because the last segment length would not be used in the transformation to absolute Cartesian coordinates.

Theta and Theta_{Step} follow a similar approach with Kappa and Kappa_{Step} respectively, but the global and relative bounds are defined over the Δ_θ values.

C. Search algorithm

We here propose a general search framework for generating roads in which the ADS under study (i.e., the ADS equipped with the lane-keeping component described in Sect. III-A) drives off the road. The framework can be instantiated with the different road representations introduced in Sect. III-B, so making it suitable to conduct the empirical study on their effectiveness. This framework is based on *Frenetic* [16], which has been shown to be effective in testing the ADS under study [33]. *Frenetic* has been adapted to use mutation operators that are not specific to one particular representation, but are applicable to all representations. We call the approach RoadSearch. We further call its instantiation with a specific road representation *rep* as RoadSearch_{rep}.

RoadSearch borrows different elements from genetic algorithms: it generates an initial population of candidates (also called *tests* in this context), it mutates previously generated tests, and it also performs crossover among those tests. The implementation of RoadSearch is shown in Listing 1. It first spends a given time budget *random_budget* (with *random_budget* < *total_budget*) for random generation (Lines 2-4), which defines the initial population of roads. Each individual is generated as explained in Sect. III-B1. Note that, whenever a test is created, it is simulated with BeamNG.tech simulator, and its simulation log is collected to compute min_{OOB} (i.e., Lines 4, 11, 15, 19, 28, and 29).

RoadSearch then selects the best candidate for mutations based on the minimum oob distance (min_{OOB}), because our hypothesis is that tests with lower min_{OOB} are more likely to produce a failure. A candidate is suitable for mutation if its min_{OOB} is lower than a given threshold (Lines 6-7). When no suitable candidate is available, the algorithm continues generating new roads randomly (Lines 18-19).

If a test is suitable for mutation, different mutators are used depending on the outcome of its simulation. Given a


```

1  def RoadSearchrep(map_size, total_budget, random_budget,
2    cross_num, cross_freq, threshold, max_visits):
3    while time < random_budget:
4      test = randomrep.generate()
5      test.minOOB = simulator.evaluate(test)
6    while time < total_budget:
7      parent = best_not_visited_candidate(threshold)
8      if parent is not None :
9        parent.visited += 1
10       if parent.passed:
11         child = alteration.apply(parent)
12         child.minOOB = simulator.evaluate(child)
13       else:
14         for mutator in [swap, reverse]:
15           child = mutator.apply(parent)
16           child.minOOB = simulator.evaluate(child)
17           child.visited = 1
18     else:
19       test = randomrep.generate()
20       test.minOOB = simulator.evaluate(test)
21     if recent_count > cross_freq:
22       parents = best_candidates(threshold, max_visits)
23       children = 0
24       while children < cross_num:
25         parent1, parent2 = rnd.choice(parents, 2)
26         if genes_similarity(parent1, parent2) < 0.95:
27           parent1.visited += 1, parent2.visited += 1
28           child1, child2 = crossover(parent1, parent2)
29           child1.minOOB = simulator.evaluate(child1)
30           child2.minOOB = simulator.evaluate(child2)
31           children += 2

```

Listing 1: RoadSearch Algorithm

passing test, alteration modifies some of the values of the test up to $\pm 10\%$ (Line 10); each gene can be altered with a given probability. In the case of tuple representation (i.e., the individual is a sequence of tuples as in Cartesian_{Cat}, Cartesian_{Bez}, Kappa_{Step}, and Theta_{Step}), the modification can be applied to one element, both, or none. Note that the goal of alteration is to drive the search towards failures.

Given a *failed* test (Line 12), the approach applies the following mutators (Line 13), which are specific to road structures: i) *swap* splits values in the middle and swaps front and back, and ii) *reverse* reverses the order of the values. The goal of using these mutators on failing tests is to possibly trigger a few additional failures.

Note that children of a *failed* test are marked as “visited” (Line 16), so that they are not eligible for further mutation, to avoid triggering failures with very similar tests.

After generating a number of mutants, the approach goes into a crossover phase among the tests with lowest *min_{OOB}* (Lines 20-30). The frequency of the crossover is defined by *cross_freq* and based on the number of tests that have been executed in the simulator before entering a crossover phase (*recent_count*). The number of children to be produced in a crossover phase is determined by *cross_num*, a parameter of the approach. To increase the diversity of the generated tests, RoadSearch has the two following constraints: (i) a test can participate in crossover up to a maximum number of visits *max_visits* (Line 21), and (ii) parents’ genes must be at least 5% different to be allowed to have children (Line 25). The similarity check is based on the number of equal genes that two candidates for crossover have. The chromosome crossover produces two children with a combination of genes from their

parents (Line 27). In this type of crossover, the position of the genes does not change and tuples are not split.

The final result of the search with RoadSearch is a *test suite* containing all the valid generated roads.

IV. EXPERIMENTS DESIGN

We here introduce the design of our study. We introduce research questions in Sect. IV-A, experimental setup in Sect. IV-B, and evaluation metrics in Sect. IV-C.

A. Research Questions (RQs)

We identified the following research questions for our study.

RQ1 Which is the *best* road representation? We will investigate this RQ in terms of two sub-research questions:

RQ1.a Which road representation allows to find more failures?

RQ1.b Which road representation produces the most diverse roads? In terms of valid generated roads (**RQ1.b.i**), and roads that produce failures (**RQ1.b.ii**).

RQ2 Which is the impact of mutations (as alteration) on different road representations? We analyze this from two perspectives, with two sub-research questions:

RQ2.a Which is the impact of alteration, regardless of the position where the mutation is done?

RQ2.b Is there a difference in impact by alteration applied in different positions of the individual?

RQ3 How different are the children produced by crossover from their parents in each representation?

B. Experimental setup

Each representation can be customized by changing its hyper-parameters. Thus, before running the experiments, we first perform a hyper-parameter study to select the best performing configuration for each road representation. The hyper-parameter study evaluates 100 different configurations for each representation. The configurations are chosen by the Tree-structured Parzen Estimator (TPE) approach [11]. The performance of each configuration is determined by a score from executing the search algorithm for 1 hour. The score is defined as $\#failures - mean(\{min_{OOB_1}, \dots, min_{OOB_n}\})$. This means that the score is mainly driven by the number of failures produced, while the mean of the minimum OOB distances is used to guide the search when no failures are triggered. Table I shows the search space of the hyper-parameters of each representation and the result of the study, i.e., the selected best hyper-parameters’ values.

Given the best configuration of hyper-parameters of each road representation, we conduct the empirical study as follows. For each road representation *rep*, we conduct a search with RoadSearch_{rep} with a total budget of 5h per experiment. The random budget for the initial population is defined as 30 minutes (10% of the total time). The size of the map is defined as 200 x 200 meters and the average length of the roads is fixed to 180m to be fair among different road representations¹. The crossover frequency (*cross_freq*) is defined

¹If road lengths were different, the number of generated tests for a given budget would largely vary.

TABLE I: Hyper-Parameters' study search space and result

representation	hyper-parameter	space (low, high, step)	selected
Cartesian _{Bez}	# search variables	(10, 50, 1)	28
	max angle (β)	(10, 90, 5)	40
	# interpolation nodes	(5, 10, 1)	9
Cartesian _{Cat}	# search variables	(10, 50, 1)	10
	max angle (β)	(10, 90, 5)	50
	# interpolation nodes	(5, 10, 1)	10
Kappa	# search variables	(10, 50, 1)	25
	global bound (gb)	(0.01, 0.1, 0.01)	0.07
	relative bound (rb)	(0.01, 0.1, 0.01)	0.05
Kappa _{Step}	# search variables	(10, 50, 1)	28
	global bound (gb)	(0.01, 0.1, 0.01)	0.08
	relative bound (rb)	(0.01, 0.1, 0.01)	0.05
	segment variance (sv)	(0.5, 10.0, 0.5)	9.5
Theta	# search variables	(10, 50, 1)	36
	global bound (gb)	($\pi/48, \pi, \pi/48$)	1/8
	relative bound (rb)	($\pi/48, \pi, \pi/48$)	1/44
Theta _{Step}	# search variables	(10, 50, 1)	19
	global bound (gb)	($\pi/48, \pi, \pi/48$)	1/5
	relative bound (rb)	($\pi/48, \pi, \pi/48$)	1/23
	segment variance (sv)	(0.5, 10.0, 0.5)	3.5

as 30, and the number of children produced in each crossover (*cross_num*) is 20. A test can participate in crossover up to 10 times (*max_visits*). The *threshold* over *min_OOB* for being considered for mutation or crossover was defined as 0.0 (i.e., the center mass of the car is on a side line).

Each experiment was repeated 30 times to account for randomness of the search algorithm, as suggested by Arcuri and Briand [4] in a guideline on conducting experiments with randomized algorithms. So, the total time for performing the experiments was 900 hours (30 repetitions $\times$ 6 road representations $\times$ 5 hours). All the experiments were run on AWS EC2 instances with 1 GPU, 4 vCPUs and 16 GB of memory (g4dn.xlarge). All the code and experimental results are available in the online repository [15].

C. Evaluation metrics

We will compare the results of RoadSearch instantiated with different road representations in terms of *number of failures* and *diversity*.

To answer RQ1.a, we compute the *number of failures* (i.e., number of tests in which the car drives off the road) produced by each approach during the search. We further analyze the source of these failures by the different methods used in the search (i.e., *random_{rep}*, *alteration*, *crossover*, *reverse*, and *swap*).

To answer RQ1.b.i, we compare all the valid generated roads in the test suite by using the *normalized discrete Procrustes-Fréchet* distance d_{NDPF} , as explained in Sect. II-B. Namely, given the n valid generated roads, we compute the *diversity* metric as the average d_{NDPF} between the generated tests:

$$diversity(t_0, \dots, t_{n-1}) = \frac{\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} d_{NDPF}(t_i, t_j)}{n \times (n-1)/2} \quad (5)$$

This means that, for each road representation, we have a distribution of 30 values. As for RQ1.b.ii, the same applies, but only to roads producing a failure, i.e., for this second analysis, we do not consider test suites that did not trigger failures.

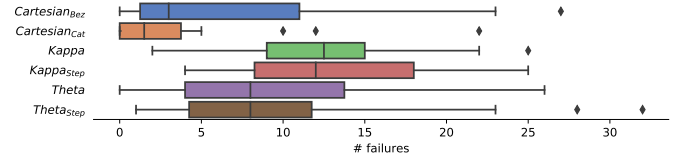


Fig. 5: RQ1.a – Failures produced by RoadSearch_{rep} for each representation *rep*

To answer RQ2, we perform alteration of a given $p\%$ to each of the genes of every randomly generated valid road. This means that we take an individual $x = [g_0, \dots, g_{n-1}]$ with n genes, modify a g_i by $p\%$ (i.e., $g'_i = g_i \pm p \times g_i$) and then compare $x' = [g_0, \dots, g'_i, \dots, g_{n-1}]$ against x for each of the positions $0, \dots, n-1$. We repeat this for $p \in \{10\%, 20\%, 30\%, 40\%, 50\%\}$. When analyzing tuples (i.e., the individual is a sequence of pairs as in Cartesian_{Cat}, Cartesian_{Bez}, Kappa_{Step}, and Theta_{Step}), we modify each of the elements of the tuple separately.

To answer RQ3, we sample 100 valid roads from the randomly generated roads and perform crossover among all of them. Each pair of parents p_1 and p_2 produce 10 children $c_0, \dots, c_9$, which are then compared against each of the parents (i.e., p_1 vs $c_0 \dots p_1$ vs c_9 and p_2 vs $c_0 \dots p_2$ vs c_9).

In the RQs, to draw more definitive conclusions, we will compare the metrics with statistical tests. Namely, given a metric M and two instances of RoadSearch, RoadSearch_{rep₁} and RoadSearch_{rep₂}, we compare the distributions of the 30 values of M using the Mann-Whitney U test² for the statistical significance (at significance level $\alpha = 0.05$), and the Vargha-Delaney's $\hat{A}_{12}$ as effect size. If Mann-Whitney U test reports a p-value less than α , we reject the null hypothesis and we claim that there is a significant difference. In this case, $\hat{A}_{12}$ indicates which approach is higher (the first one if $\hat{A}_{12}$ is greater than 0.5, the second one otherwise) and the strength of the difference: in case the first (resp. second) is greater, the strength is *negligible* if $\hat{A}_{12} \in (0.5, 0.556)$ (resp. $\in (0.494, 0.5)$), *small* if $\hat{A}_{12} \in [0.556, 0.638)$ (resp. $\in (0.362, 0.494]$), *medium* if $\hat{A}_{12} \in [0.638, 0.714)$ (resp. $\in (0.286, 0.362]$), and *large* if $\hat{A}_{12} \geq 0.714$ (resp. ≤ 0.286).

V. EXPERIMENTAL RESULTS

A. RQ1.a – Failure detection

In this RQ, we are interested in investigating which is the impact of the road representation on the effectiveness of the search algorithm, i.e., its ability to produce failures. Fig. 5 shows the number of failures (i.e., the car driving off the road) produced by each representation, in each of the 30 repetitions of the experiment. Note that some test suites did not produce any failure, namely 8 for Cartesian_{Bez}, 15 for Cartesian_{Cat}, 4 for Theta, and 1 for Theta_{Step}.

To draw more significant conclusions, we performed a Mann-Whitney U-test and $\hat{A}_{12}$ statistic between the results of

²Note that, in general, the compared values are not normally distributed; for this reason, we use a non-parametric test.

TABLE II: RQ1.a – Failures produced by RoadSearch_{rep} for each representation *rep* – Comparison between the representations (Legend. ✓: first approach is significantly better; ✗ second approach is significantly better; ≡: there is no significant difference. $\hat{A}_{12}$ value reported in parentheses)

	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	✓ (0.677)	✗ (0.249)	✗ (0.231)	≡	✗ (0.326)
Cartesian _{Cat}	✗ (0.323)	-	✗ (0.0744)	✗ (0.0661)	✗ (0.204)	✗ (0.138)
Kappa	✓ (0.751)	✓ (0.926)	-	≡	✓ (0.661)	✓ (0.676)
Kappa _{Step}	✓ (0.769)	✓ (0.934)	≡	-	✓ (0.659)	✓ (0.669)
Theta	≡	✓ (0.796)	✗ (0.339)	✗ (0.341)	-	≡
Theta _{Step}	✓ (0.674)	✓ (0.862)	✗ (0.324)	✗ (0.331)	≡	-

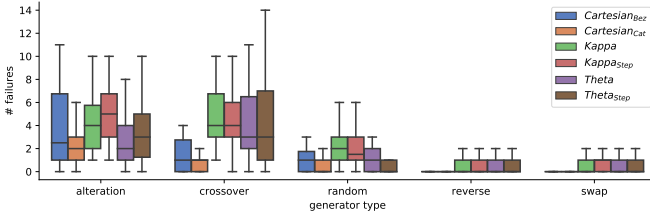


Fig. 6: RQ1.a – Failures produced by RoadSearch_{rep} for each representation *rep* – Results by generator type

each pair of approaches, as described in Sect. IV-C. Table II reports the results of the statistical tests. In the table, ✓ means that the first approach is significantly better than the second one, while ✗ means the opposite; ≡ means that there is no significant difference. The value between parentheses is the $\hat{A}_{12}$ value. The results show that Kappa and Kappa_{Step} triggered more failures than all the other approaches, but the difference between them is not significant. Moreover, Theta and Theta_{Step} are better (equal in one case) than the Cartesian representations Cartesian_{Bez} and Cartesian_{Cat}.

Fig. 6 illustrates the source of the failures for each representation. The results show that alteration and crossover are the main source of failures. We also compared the different approaches on these results with the statistical test, as reported in Table III. In some cases, results are similar to those of the aggregated results (i.e., Fig. 5 and Table II), but there are notable differences across the different operators. We observe that Kappa and Kappa_{Step} produce significantly more failures than all the other approaches by using alteration and random_{rep}, but there is no significant difference with Theta and Theta_{Step} regarding crossover, swap, and reverse. As for Cartesian_{Bez}, the results show that it produces significantly less failures than every other non-Cartesian representation, except for value alteration and reverse when compared to Theta and Theta_{Step}, and for random_{rep} when compared to Theta.

B. RQ1.b – Diversity analysis

In this RQ, we are interested in assessing the diversity of the generated test suites, as higher diversity is an indication of a better exploration of the search space, and so possibly a higher effectiveness. Regarding RQ1.b.i, Fig. 7 shows, for each representation, the *diversity* metric (see Eq. 5) of each

TABLE III: RQ1.a – Failures produced by RoadSearch_{rep} for each representation *rep* – Results by generator type – Comparison between the representations (Legend. ✓: first approach is significantly better; ✗ second approach is significantly better; ≡: there is no significant difference. $\hat{A}_{12}$ value reported in parentheses)

alteration	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	✓ (0.626)	✗ (0.355)	✗ (0.327)	≡	≡
Cartesian _{Cat}	✗ (0.374)	-	✗ (0.183)	✗ (0.152)	✗ (0.367)	✗ (0.272)
Kappa	✓ (0.645)	✓ (0.817)	-	≡	✓ (0.721)	≡
Kappa _{Step}	✓ (0.673)	✓ (0.848)	≡	-	✓ (0.766)	✓ (0.662)
Theta	≡	✓ (0.633)	✗ (0.279)	✗ (0.234)	-	≡
Theta _{Step}	≡	✓ (0.728)	≡	✗ (0.338)	≡	-
crossover	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	≡	✗ (0.207)	✗ (0.239)	✗ (0.309)	✗ (0.262)
Cartesian _{Cat}	≡	-	✗ (0.079)	✗ (0.129)	✗ (0.209)	✗ (0.147)
Kappa	✓ (0.793)	✓ (0.921)	-	≡	≡	≡
Kappa _{Step}	✓ (0.761)	✓ (0.871)	≡	-	≡	≡
Theta	✓ (0.691)	✓ (0.791)	≡	≡	-	≡
Theta _{Step}	✓ (0.738)	✓ (0.853)	≡	≡	≡	-
random_{rep}	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	✓ (0.685)	✗ (0.249)	✗ (0.318)	≡	✓ (0.630)
Cartesian _{Cat}	✗ (0.315)	-	✗ (0.146)	✗ (0.183)	✗ (0.322)	≡
Kappa	✓ (0.751)	✓ (0.854)	-	≡	✓ (0.726)	✓ (0.833)
Kappa _{Step}	✓ (0.682)	✓ (0.817)	≡	-	✓ (0.663)	✓ (0.783)
Theta	≡	✓ (0.678)	✗ (0.274)	✗ (0.337)	-	✓ (0.627)
Theta _{Step}	✗ (0.370)	≡	✗ (0.167)	✗ (0.217)	✗ (0.373)	-
reverse	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	✓ (0.583)	✗ (0.356)	✗ (0.379)	≡	≡
Cartesian _{Cat}	✗ (0.417)	-	✗ (0.283)	✗ (0.300)	✗ (0.350)	✗ (0.350)
Kappa	✓ (0.644)	✓ (0.717)	-	≡	≡	≡
Kappa _{Step}	✓ (0.621)	✓ (0.700)	≡	-	≡	≡
Theta	≡	✓ (0.650)	≡	≡	-	≡
Theta _{Step}	≡	✓ (0.650)	≡	≡	≡	-
swap	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	≡	✗ (0.305)	✗ (0.256)	✗ (0.350)	✗ (0.336)
Cartesian _{Cat}	≡	-	✗ (0.283)	✗ (0.233)	✗ (0.333)	✗ (0.317)
Kappa	✓ (0.695)	✓ (0.717)	-	≡	≡	≡
Kappa _{Step}	✓ (0.744)	✓ (0.767)	≡	-	≡	≡
Theta	✓ (0.650)	✓ (0.667)	≡	≡	-	≡
Theta _{Step}	✓ (0.664)	✓ (0.683)	≡	≡	≡	-

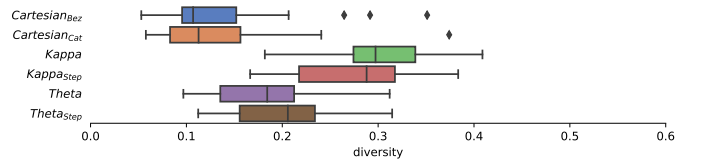


Fig. 7: RQ1.b.i – Diversity of tests in a test suite

generated test suite in the 30 repetitions. Table IV shows the results of the comparison with statistical test between the different representations. The results show that Kappa and Kappa_{Step} produce more diverse tests than the other representations; moreover, Kappa is better than Kappa_{Step}. Next, we have Theta and Theta_{Step}, while Cartesian_{Bez} and Cartesian_{Cat} have the least diverse tests.

Fig. 8 shows a similar analysis focused on the diversity of the roads that produce failures (RQ1.b.ii). Note that here not all the representations have the same number of data points because some test suites did not produce any failure,

TABLE IV: RQ1.b.i – Diversity of tests in a test suite – Comparison between the representations

	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	≡	✗ (0.048)	✗ (0.070)	✗ (0.242)	✗ (0.177)
Cartesian _{Cat}	≡	-	✗ (0.033)	✗ (0.048)	✗ (0.203)	✗ (0.142)
Kappa	✓ (0.952)	✓ (0.967)	-	✓ (0.636)	✓ (0.924)	✓ (0.913)
Kappa _{Step}	✓ (0.930)	✓ (0.952)	✗ (0.364)	-	✓ (0.853)	✓ (0.789)
Theta	✓ (0.758)	✓ (0.797)	✗ (0.076)	✗ (0.147)	-	≡
Theta _{Step}	✓ (0.823)	✓ (0.858)	✗ (0.087)	✗ (0.211)	≡	-

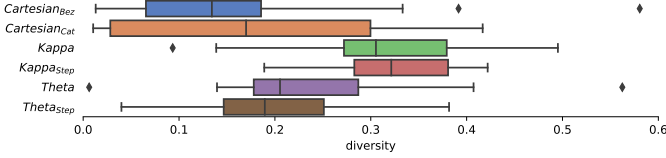


Fig. 8: RQ1.b.ii – Diversity of failing tests in a test suite

TABLE V: RQ1.b.ii – Diversity of failing tests in a test suite – Comparison between the representations

	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	≡	✗ (0.152)	✗ (0.121)	✗ (0.255)	✗ (0.318)
Cartesian _{Cat}	≡	-	✗ (0.267)	✗ (0.249)	≡	≡
Kappa	✓ (0.848)	✓ (0.733)	-	≡	✓ (0.738)	✓ (0.807)
Kappa _{Step}	✓ (0.879)	✓ (0.751)	≡	-	✓ (0.795)	✓ (0.851)
Theta	✓ (0.745)	≡	✗ (0.262)	✗ (0.205)	-	≡
Theta _{Step}	✓ (0.682)	≡	✗ (0.193)	✗ (0.149)	≡	-

as explained in Sect. V-A. Overall, we can observe that the diversity of these subset of tests is slightly higher than in the entire test suite; this is a good indication that the search approach can not only explore diverse roads, but can also provide diverse final solutions: this is important for ADS debugging, in which ADS engineers may want to observe the ADS failure under different perspectives. As in RQ1.b.i, Kappa and Kappa_{Step} show the most diverse test suites. Cartesian_{Bez} performance is the worst in terms of diversity, while Cartesian_{Cat} diversity is similar to that of Theta and Theta_{Step}, but not better than Cartesian_{Bez}. See Table V for the details of the statistical comparison.

C. R2.a – Sensitivity to different alteration levels

The effect of a mutator can be different for each representation, so producing more or less diverse tests. We here consider the alteration mutation, which is the most typical mutator used in search algorithms (together with crossover, investigated in RQ3). Fig. 9 shows the impact of different levels p of alteration ($p \in \{10\%, 20\%, 30\%, 40\%, 50\%\}$) on different road representations. Namely, it reports the distance d_{NDPF} between the original test and the altered test. The results show that Cartesian_{Cat} is more sensitive to alteration than all the other representations, while Cartesian_{Bez} and Theta are the least sensitive. Increasing the level of alteration does not change the order of sensitivity between representations, but increases the differences in all the representations. Note that Cartesian_{Cat} is the most sensitive to alteration, but the one producing less failures by using this method (see RQ1.a).

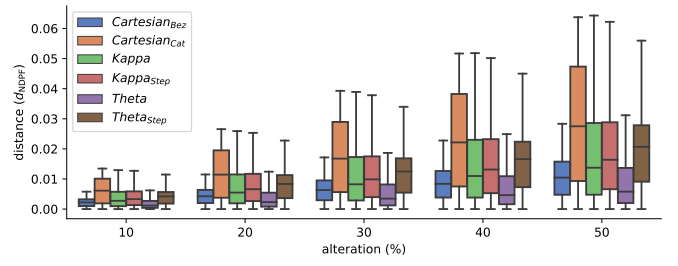


Fig. 9: RQ2.a – Impact of alteration in different representations in terms of d_{NDPF}

TABLE VI: RQ2.a – Impact of 10% alteration in different representations – Comparison between the representations

	Cartesian _{Bez}	Cartesian _{Cat}	Kappa	Kappa _{Step}	Theta	Theta _{Step}
Cartesian _{Bez}	-	✗ (0.252)	✗ (0.399)	✗ (0.352)	✓ (0.607)	✗ (0.251)
Cartesian _{Cat}	✓ (0.748)	-	✓ (0.646)	✓ (0.629)	✓ (0.781)	✓ (0.624)
Kappa	✓ (0.601)	✗ (0.354)	-	✗ (0.472)	✓ (0.669)	✗ (0.428)
Kappa _{Step}	✓ (0.648)	✗ (0.371)	✓ (0.528)	-	✓ (0.702)	✗ (0.455)
Theta	✗ (0.393)	✗ (0.219)	✗ (0.331)	✗ (0.298)	-	✗ (0.241)
Theta _{Step}	✓ (0.749)	✗ (0.376)	✓ (0.572)	✓ (0.545)	✓ (0.759)	-

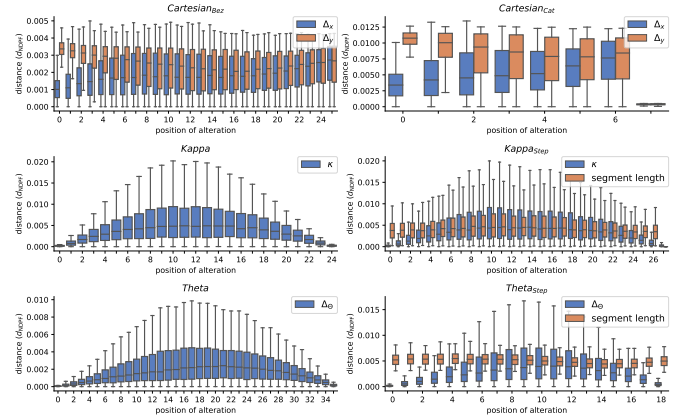


Fig. 10: RQ2.b – Impact of 10% alteration with respect to the position in terms of distance (as d_{NDPF})

The reason is that some of those modified roads become too sharp and thus invalid. Table VI reports the statistical comparison between the different representations (for the 10% alteration), which shows that there is significant difference among all the representations. Statistical results are the same for all the alteration levels, and so not reported here.

D. R2.b – Impact of alterations in different positions

In a classical search algorithm, alteration can be applied to any gene with equal probability. However, in our setting, it could be that alterations in different positions have different impact on the modification of the road; in this RQ, we investigate this aspect. Fig. 10 shows the impact of 10% alteration to each of the different positions of these representations. The results show that, for Cartesian_{Cat} and Cartesian_{Bez}, modifications to Δ_x of a control node seem to be less significant than those to Δ_y in the first positions of

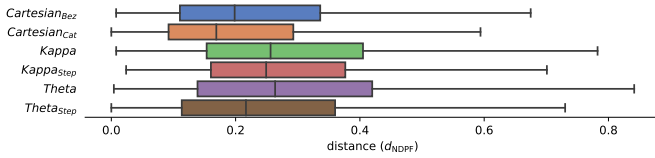


Fig. 11: RQ3 – Distance (as d_{NDPF}) between parents and children obtained with the `crossover` operator

the individual, while the difference becomes less significant towards the end. This is because each control node can not deviate more than a given maximum angle (max_angle) w.r.t. the previous control node (40° in $Cartesian_{Bez}$ and 50° in $Cartesian_{Cat}$). Since the generation starts along the y axis, the distance between the initial control nodes is higher along the y axis than the x axis. The road can start distributing more along the x axis only after a given number of control nodes. For Kappa and Kappa_{Step}, we observe that modifications to kappa values in the middle of the individual seem to be more significant than the modifications to the extreme positions. The same conclusion applies to theta values of Theta and Theta_{Step}. This follows the intuition that bending a curve closer to the center would generate a larger difference. As for the segment length of Kappa_{Step} and Theta_{Step}, we do not observe a significant difference regarding the position.

E. RQ3 – Sensitivity to Crossover

We also compared the impact of the crossover operator on the different road representations. Namely, we applied the `crossover` operator on the randomly generated tests and we computed the distance d_{NDPF} between each parent and each generated child. Fig. 11 illustrates, for each road representation, the distribution of d_{NDPF} for all the comparisons. We also compared the distributions of d_{NDPF} with statistical tests, and we found that there is no significant difference across the road representation, and so we avoid reporting the results. This shows that `crossover` is not sensitive to the road representation.

VI. THREATS TO VALIDITY

The validity of our study may be affected by some threats. We discuss them, following the classical classification of *construct*, *conclusion*, *internal*, and *external validities* [37].

Construct validity: A threat of this type could be that the metrics used in the experiments do not reflect the object of the investigation, i.e., the influence of the road representation on the effectiveness of SBT for ADSs. As first metric, we consider the number of failing tests found by the search algorithm instantiated with different road representations; as such, this metric directly reflects the objective of the search, and so it is a suitable metric. Moreover, we have also compared the generated roads in terms of their *diversity*, based on the normalized discrete Procrustes-Fréchet distance (d_{NDPF}); producing different roads allows to explore the search space more, and so possibly to better optimize the objective function.

Thus, we also believe that this metric is suitable to assess the impact of the road representation on the effectiveness of SBT. Note that we have also tried other distance metrics (e.g., relative angle distance) to assess diversity and all give results similar to those we obtained; this gives us confidence on the quality of the metric.

Conclusion validity: Different factors can affect the ability to draw definitive conclusions. A search algorithm is affected by some random behavior and this should be considered in the evaluation. To this aim, each experiment (i.e., execution of RoadSearch instantiated with each road representation) has been repeated for 30 times, as suggested by Arcuri and Briand [4] in their guideline on conducting experiments with randomized algorithms. Still following [4], to draw significant conclusions, we have compared the results of the different representations using statistical tests that account both for statistical significance and effect size. Values of the hyper-parameters of the different representations can affect the effectiveness of the search; so, to guarantee a fair comparison, we performed a hyper-parameter study to select the best performing configuration for each representation (see Sect. IV-B).

Internal validity: A threat of this type could be that the relationship between the used road representation and the obtained results is by chance. For example, a faulty implementation of some representation could lead to often generate invalid roads or to not allow the vehicle to move, so reducing the possibility of finding valid tests in which the car drives off the road. To mitigate this threat, we have carefully tested the implementation, and we have implemented measures to reduce the number of invalid roads produced (see Sect. III-B1) to discard those that are invalid.

External validity: An external validity threat is that the results may not be generalizable. First of all, in the experiments we have generated driving scenarios only constituted of the road, without other traffic participants (i.e., other cars and pedestrian), regulatory elements such as traffic lights, surrounding elements such as buildings, etc. Such simple type of scenario is suitable for our analysis, since it allows to better assess the relationship between the type of road and the performance of SBT, as there are no other confounding factors. However, it could be that the combination of the road representation and another scenario element could improve the search. We will further investigate this in our future work. Moreover, our experiments have been conducted for the testing of a particular ADS component (i.e., the lane-keeping component) that is affected by the road representation. However, there are other goals in ADS testing that can depend on the type of road (e.g., the generation of collision scenarios), and it is not clear whether our findings also apply in those contexts. We also leave this type of analysis as future work.

VII. RELATED WORK

In this section, we review works related to our study.

Search-Based Testing (SBT) [30] is a popular testing approach, that can effectively search in the space of inputs of a system under test, guided by an optimization function

related to a given test requirement. Notable SBT approaches are unit test generation for Java code with EvoSuite [21], system level testing for RESTful APIs with EvoMaster [3], and test generation and minimization for Android applications with Sapienz [29].

In search algorithms, the problem of an efficient encoding (i.e., genotype) for representing the target of the search (i.e., the phenotype) is widely studied [19], [38]. This problem is relevant also in SBT. For example, Sapienz introduced the notion of *motifs*, that represent *usage patterns* of Android apps (i.e., sequences of atomic actions), and used these for the representation of an individual, instead of atomic actions. Such representation has been proven to be the main reason of the effectiveness of the approach [31].

In our work, we consider SBT for Autonomous Driving Systems (ADSs); several works have been proposed in this domain, thanks to the ability of the search to explore the complex and large search space of driving scenarios for ADSs. For example, Ben Abdesslem et al. [8] test a Pedestrian Detection Vision system using multi-objective search; in order to tackle the problem of expensive ADS simulations, they employ *surrogate models* to approximate the ADS behavior. Ben Abdesslem et al. [10] use a search approach to find scenarios in which the different components of the ADS (e.g., the Autonomous Cruise Control and the Pedestrian Protection) interact in an undesired way, leading to failure. Calò et al. [13], [14], to test an ADS path planner, use a search approach to find collision scenarios that can be proven to be *avoidable*, i.e., in which the path planner configured in a different way does not lead to a collision. The same path planner is also considered by Arcaini et al. [2] who use a search approach to find scenarios in which the vehicle performs specific driving maneuvers, e.g., turning with high later acceleration.

All previous works do not search for the road structure, that is instead fixed in advance, but only for other characteristics of the scenarios, such as acceleration and position of other vehicles, initial position of the autonomous vehicle, etc. Ben Abdesslem et al. [9] consider the road structure as part of the search, but in a coarse manner by choosing the road topology from a subset of fixed road topologies (e.g., straight, curved, or ramped) that can be only lightly tuned. Only recently, the SBT community started to specifically investigate on search for complete roads. Gambi et al. [22] use procedural content generation and search-based testing to also find scenarios that produce out of bound errors (i.e., makes the car go drive off the lane). The procedural generation is done by composing road segments created with polylines (e.g., “straight segment 30m” or “20° turn”); we note that this procedural generation technique can be described through Θ_{Step} representation by modifying the numerical integration scheme (from trapezoidal integration to rectangular integration). Their results show that using SBT and procedural content generation seems to be more effective than searching scenarios randomly. Riccio and Tonella [34] propose a search-based tool that generates frontier inputs for DL systems. In their work, they also focus on searching single-lane roads to

test the lane-keeping component of an ADS and use Catmull-Rom cubic splines as their representation of roads. Klück et al. [24] propose using genetic algorithms (GA) and Bézier curves as the road representation for single-lane roads. Their results showed that their approach produces several failures, but the failing test cases have high similarity, which reduces the overall efficiency of the technique. However, it is unclear if it is due to GAs or the representation. In our previous work, Castellano et al. [16] proposed *Frenetic*, a genetic-like algorithm for searching single-lane roads that uses curvature values of Frenet frames as its representation. Compared to standard GAs, this technique adds some nuances with the aim of improving the diversity of the generated test.

The previous works [22], [34], [24], [16] use different road representations and different search algorithms to test the same lane-keeping component. Therefore, it is not possible to directly use those techniques to assess the influence of road representation on the effectiveness of SBT for ADSs, as it would not be clear if the results are due to the search algorithm or to the representation. Thus, in this work, to investigate the effectiveness of road representation, we have compared a set of representative road representations by using the same search algorithm, so to make the comparison fair.

VIII. CONCLUSIONS AND FUTURE WORK

In this work, we compared six different road representations when testing the lane-keeping component of an ADS. Overall, using Cartesian-based representations (Cartesian_{Cat} and Cartesian_{Bez}) seems to be less effective than using nonparametric representations, such as those based on curvatures or orientation. These latter representations (Kappa , Kappa_{Step} , Theta , and Theta_{Step}) seem to be more suitable for testing ADS, since they are especially targeted towards driving applications; indeed, the performance of an automated driving system largely depends on the sharpness and the position of the turns along the road, which can be better tuned with curvatures and orientation based representations.

In particular, our experimental results show that using curvature-based road representations (Kappa and Kappa_{Step}) triggers more failures and that the generated tests are more diverse. Experiments also show that alteration on different positions have different impact depending on the representation, which may also affect the search effectiveness. The impact of crossover, instead, is similar across representations.

In our future work, we plan to increase the complexity of scenarios by introducing new traffic participants, such as other cars, pedestrians, etc. Having other traffic participants will allow us to also extend our analysis to different type of search in which the shape of the road may also play an important role (e.g., when searching for collisions).

ACKNOWLEDGMENT

The authors are supported by ERATO HASUO Metamathematics for Systems Design Project (No. JPMJER1603), JST. Funding reference number: 10.13039/501100009024 ERATO.

A. Cetinkaya is also supported by JSPS Kakenhi Grant Number 20K14771.

REFERENCES

- [1] P. K. Agarwal, R. B. Avraham, H. Kaplan, and M. Sharir. Computing the discrete Fréchet distance in subquadratic time. *SIAM Journal on Computing*, 43(2):429–449, 2014.
- [2] P. Arcaini, X.-Y. Zhang, and F. Ishikawa. Targeting patterns of driving characteristics in testing autonomous driving systems. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 295–305, 2021.
- [3] A. Arcuri. EvoMaster: Evolutionary multi-context automated system test generation. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, pages 394–397, 2018.
- [4] A. Arcuri and L. Briand. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering, ICSE '11*, pages 1–10, New York, NY, USA, 2011. ACM.
- [5] B. Aronov, S. Har-Peled, C. Knauer, Y. Wang, and C. Wenk. Fréchet distance for curves, revisited. In *European symposium on algorithms*, pages 52–63, 2006.
- [6] BeamNG.tech. Beamng tech. <https://beamng.tech/>. Accessed: 08.2021.
- [7] A. Belbachir, J.-C. Smal, J.-M. Blossville, and D. Gruyer. Simulation-driven validation of advanced driving-assistance systems. *Procedia - Social and Behavioral Sciences*, 48:1205–1214, 2012.
- [8] R. Ben Abdesslem, S. Nejati, L. C. Briand, and T. Stifter. Testing advanced driver assistance systems using multi-objective search and neural networks. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, ASE 2016*, pages 63–74, New York, NY, USA, 2016. ACM.
- [9] R. Ben Abdesslem, S. Nejati, L. C. Briand, and T. Stifter. Testing vision-based control systems using learnable evolutionary algorithms. In *Proceedings of the 40th Int. Conference on Software Engineering, ICSE '18*, pages 1016–1026, New York, NY, USA, 2018. ACM.
- [10] R. Ben Abdesslem, A. Panichella, S. Nejati, L. C. Briand, and T. Stifter. Testing autonomous cars for feature interaction failures using many-objective search. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE 2018*, pages 143–154, New York, NY, USA, 2018. ACM.
- [11] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl. Algorithms for hyperparameter optimization. *Advances in neural information processing systems*, 24, 2011.
- [12] O. Bühler and J. Wegener. Evolutionary functional testing of a vehicle brake assistant system. In *6th Metaheuristics International Conference, Vienna, Austria*, 2005.
- [13] A. Calò, P. Arcaini, S. Ali, F. Hauer, and F. Ishikawa. Generating avoidable collision scenarios for testing autonomous driving systems. In *2020 IEEE 13th International Conference on Software Testing, Verification and Validation (ICST)*, pages 375–386, 2020.
- [14] A. Calò, P. Arcaini, S. Ali, F. Hauer, and F. Ishikawa. Simultaneously searching and solving multiple avoidable collisions for testing autonomous driving systems. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference, GECCO '20*, pages 1055–1063, New York, NY, USA, 2020. Association for Computing Machinery.
- [15] E. Castellano, A. Cetinkaya, and P. Arcaini. Analysis of road representations in search-based testing of autonomous driving systems – additional material. <https://github.com/ERATOMMSD/roadsearch>, 2021.
- [16] E. Castellano, A. Cetinkaya, C. H. Thanh, S. Klikovits, X. Zhang, and P. Arcaini. Frenetic at the SBST 2021 tool competition. In *2021 IEEE/ACM 14th International Workshop on Search-Based Software Testing (SBST)*, pages 36–37, 2021.
- [17] E. Catmull and R. Rom. A class of local interpolating splines. In *Computer Aided Geometric Design*, pages 317–326. Academic Press, 1974.
- [18] I. L. Dryden and K. V. Mardia. *Statistical Shape Analysis: With Applications in R*. John Wiley & Sons, 2016.
- [19] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Springer Publishing Company, Incorporated, 2nd edition, 2015.
- [20] @elonbachman. Tesla deaths, Feb. 2020. Regularly updated at tesladeaths.com; version hosted on Zenodo will be updated periodically.
- [21] G. Fraser and A. Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, ESEC/FSE '11*, pages 416–419, New York, NY, USA, 2011. Association for Computing Machinery.
- [22] A. Gambi, M. Mueller, and G. Fraser. Automatically testing self-driving cars with search-based procedural content generation. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, pages 318–328, New York, NY, USA, 2019. Association for Computing Machinery.
- [23] N. Kalra and S. M. Paddock. Driving to safety: How many miles of driving would it take to demonstrate autonomous vehicle reliability? *Transportation Research Part A: Policy and Practice*, 94:182–193, 2016.
- [24] F. Klück, L. Klampfl, and F. Wotawa. GABezier at the SBST 2021 tool competition. In *2021 IEEE/ACM 14th International Workshop on Search-Based Software Testing (SBST)*, pages 38–39, 2021.
- [25] S. Kobayashi. *Differential Geometry of Curves and Surfaces*. Springer, 2019.
- [26] T. Laurent, P. Arcaini, F. Ishikawa, and A. Ventresque. Achieving weight coverage for an autonomous driving system with search-based test generation. In *2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, Oct 2020.
- [27] S. Levin and J. Wong Carrie. Self-driving Uber kills Arizona woman in first fatal crash involving pedestrian. <https://www.theguardian.com/technology/2018/mar/19/uber-self-driving-car-kills-woman-arizona-tempe>, Mar 2018. Last accessed: August 18, 2021.
- [28] G. Li, Y. Li, S. Jha, T. Tsai, M. B. Sullivan, S. K. S. Hari, Z. Kalbarczyk, and R. K. Iyer. AV-FUZZER: Finding safety violations in autonomous driving systems. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, pages 25–36, 2020.
- [29] K. Mao, M. Harman, and Y. Jia. Sapienz: Multi-objective automated testing for Android applications. In *Proceedings of the 25th International Symposium on Software Testing and Analysis, ISSTA 2016*, pages 94–105, New York, NY, USA, 2016. Association for Computing Machinery.
- [30] P. McMinn. Search-based software testing: Past, present and future. In *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*, pages 153–163, 2011.
- [31] I. A. Moreno, J. P. Galeotti, and D. Garbervetsky. Algorithm or representation? an empirical study on how SAPIENZ achieves coverage. In *Proceedings of the IEEE/ACM 1st International Conference on Automation of Software Test, AST '20*, pages 61–70, New York, NY, USA, 2020. Association for Computing Machinery.
- [32] A. Mosig and M. Clausen. Approximately matching polygonal curves with respect to the Fréchet distance. *Computational Geometry*, 30(2):113–127, 2005.
- [33] S. Panichella, A. Gambi, F. Zampetti, and V. Riccio. SBST tool competition 2021. In *2021 IEEE/ACM 14th International Workshop on Search-Based Software Testing (SBST)*, pages 20–27, 2021.
- [34] V. Riccio and P. Tonella. Model-based exploration of the frontier of behaviours for deep learning system testing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020*, pages 876–888, New York, NY, USA, 2020. Association for Computing Machinery.
- [35] T. E. Vos, F. F. Lindlar, B. Wilmes, A. Windisch, A. I. Baars, P. M. Kruse, H. Gross, and J. Wegener. Evolutionary functional black-box testing in an industrial setting. *Software Quality Journal*, 21(2):259–288, 2013.
- [36] M. Werling, J. Zieglerand, S. Kammel, and S. Thrun. Optimal trajectory generation for dynamic street scenarios in a Frenet frame. In *Proc. IEEE Int. Conf. Robot. Autom.*, pages 987–993, 2010.
- [37] C. Wohlin, P. Runeson, M. Hst, M. C. Ohlsson, B. Regnell, and A. Wessln. *Experimentation in Software Engineering*. Springer Publishing Company, Incorporated, 2012.
- [38] X. Yu and M. Gen. *Introduction to Evolutionary Algorithms*. Springer Publishing Company, Incorporated, 2012.
- [39] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda. A survey of autonomous driving: Common practices and emerging technologies. *IEEE Access*, 8:58443–58469, 2020.
- [40] S. Zhu and B. Aksun-Guvenc. Trajectory planning of autonomous vehicles based on parameterized control optimization in dynamic on-road environments. *J. Intel. Robot. Syst.*, 100(3):1055–1067, 2020.